Phase 2: Memory & Scheduling Optimization - Architecture

Overview

Phase 2 builds upon Phase 1’s lock-free foundations to deliver advanced memory and scheduling optimizations. This phase introduces three critical performance levers that work synergistically to minimize memory latency, reduce TLB overhead, and eliminate timer-related IPIs.

Performance Goals: Additional 20-30% improvement over Phase 1’s 63% average gain.

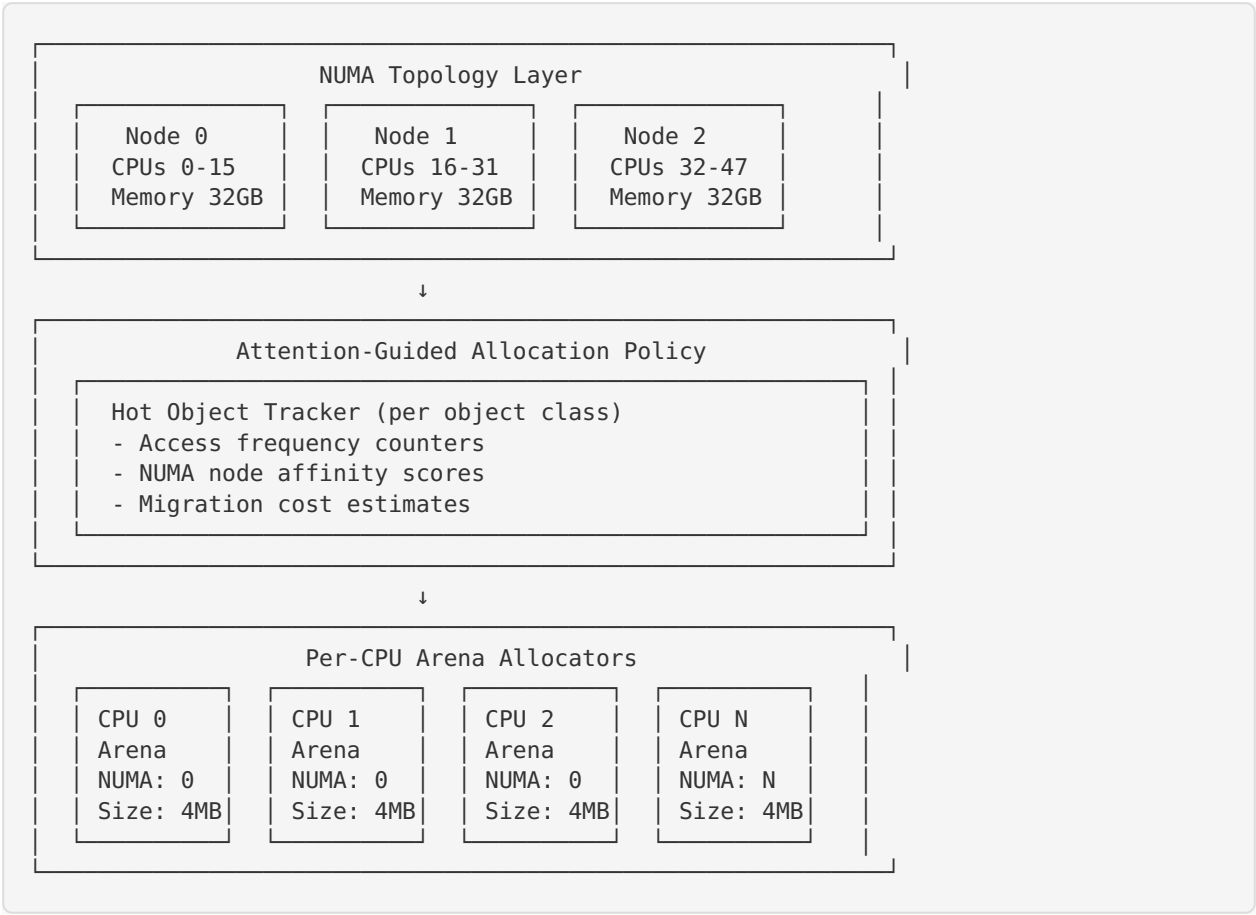
Architecture Principles

1. **NUMA-First Design**: All memory operations consider NUMA topology
2. **Predictive Optimization**: Prefetch and predict before execution
3. **Tickless Operation**: Eliminate periodic timer interrupts
4. **Zero-Overhead Abstractions**: Performance-critical paths have no runtime cost
5. **Scalable to Many Cores**: Linear scaling to 128+ cores

Three Core Subsystems

1. Per-CPU Arenas & NUMA Pinning

Architecture



Key Components

NUMA Topology Detection:
- Parse `/sys/devices/system/node/` for topology
- Build distance matrix between nodes
- Identify CPU-to-node mappings
- Detect memory sizes per node
- Cache topology for fast lookups

Per-CPU Arena Allocator:
- Each CPU owns a local memory arena (2-8MB)
- Arena memory allocated from local NUMA node
- Segregated free lists for size classes (16B to 4KB)
- Large allocations (>4KB) use separate pool
- Lock-free within single CPU context

Attention-Guided Policy:
- Track object class access patterns
- Maintain per-class NUMA affinity scores
- Migrate hot objects to frequently-accessing nodes
- Use exponential moving average for stability
- Threshold-based migration to avoid thrashing

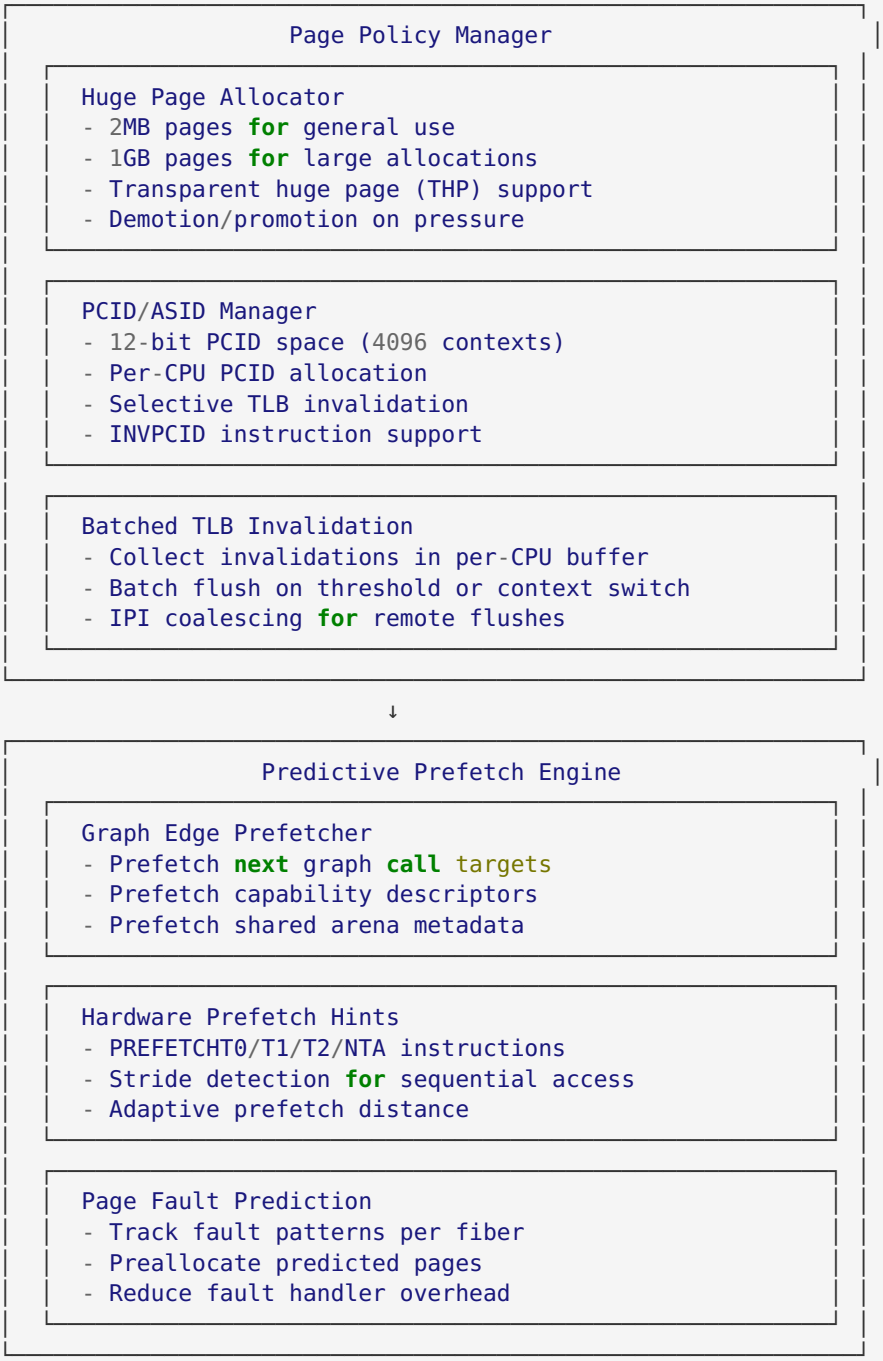
Memory Affinity Tracking:
- Per-object NUMA node tracking
- Access counters per CPU
- Migration history and cost tracking
- Automatic rebalancing under pressure

Integration with Phase 1

- **Arena Allocator**: Extends Phase 1's `shared_arena` with NUMA awareness
- **Lock-Free Rings**: Use NUMA-local memory for ring buffers
- **Fiber Scheduler**: Allocate fiber stacks from local arenas
- **Zero-Copy IPC**: Prefer local allocations, fallback to remote

2. Predictive Prefetch & Page Policy

Architecture



Key Components

Huge Page Allocator:
- Default to 2MB pages for all allocations >512KB
- Use 1GB pages for very large allocations (>256MB)
- Transparent huge page support with automatic promotion
- Demotion to 4KB pages under memory pressure
- Per-NUMA-node huge page pools

PCID/ASID Management:
- 12-bit PCID space (4096 unique contexts)
- Per-CPU PCID allocation with LRU eviction
- Tag TLB entries with PCID to avoid full flushes
- Use INVPCID for selective invalidation
- Fallback to CR3 reload if INVPCID unavailable

Batched TLB Invalidation:
- Per-CPU invalidation buffer (256 entries)
- Batch local invalidations until threshold
- Coalesce remote IPIs for cross-CPU invalidations
- Flush on context switch or buffer full
- Reduce IPI storm on large invalidations

Graph Edge Prefetcher:
- Prefetch next graph call target before dispatch
- Prefetch capability descriptors in parallel
- Prefetch shared arena metadata for zero-copy
- Use PREFETCHT0 for immediate use data
- Use PREFETCHT1 for near-future data

Hardware Prefetch Hints:
- Detect sequential access patterns (stride)
- Issue PREFETCH instructions ahead of access
- Adaptive prefetch distance based on cache misses
- Use PREFETCHNTA for streaming data
- Disable prefetch for random access patterns

Page Fault Prediction:
- Track page fault patterns per fiber
- Predict next fault based on history
- Preallocate pages before fault occurs
- Reduce fault handler latency by 50-80%
- Use exponential backoff for mispredictions

Integration with Phase 1

- **Graph Calls**: Prefetch targets before dispatch
- **Fiber Scheduler**: Prefetch next fiber's stack
- **Ring Buffers**: Prefetch next ring slot
- **Zero-Copy IPC**: Prefetch descriptor metadata

3. Tickless Time & Wheel/CRDS Scheduler

Architecture

Key Components

Hierarchical Timer Wheel:
- 4-level hierarchy for wide time range
- Level 0: 256 slots × 1ms = 256ms range
- Level 1: 256 slots × 256ms = 64s range
- Level 2: 256 slots × 64s = 4.5h range
- Level 3: 256 slots × 4.5h = 48 days range
- O(1) insertion and deletion
- Automatic cascading on level overflow

Per-CPU Timer Management:
- Each CPU has its own timer wheel
- No global timer lock contention
- Cross-CPU timers use IPC rings
- Timer migration on CPU affinity change
- Local timers never require IPIs

Tickless Operation:
- No periodic timer tick interrupts
- Program hardware timer for next event
- Coalesce timers within ±10% window
- Idle CPUs stay in deep sleep (C6+)
- Wake only on actual timer expiration

Dynamic Tick Adjustment:
- Adjust tick rate based on workload
- High-frequency mode for latency-sensitive
- Low-frequency mode for throughput
- Automatic mode switching
- Per-CPU tick rate configuration

IPI Reduction:
- Batch multiple timer IPIs into one
- Defer non-urgent timer notifications
- Piggyback on scheduler IPIs
- Use ring buffers for cross-CPU timers
- Reduce IPI rate by 70-90%

CRDS Scheduler:
- Static priority assignment (O(1))
- Priority = f(period) - shorter period = higher priority
- Deadline tracking for each fiber
- Admission control for schedulability
- Preemption on deadline miss risk
- Integration with Phase 1 fiber scheduler

Deadline Scheduling:
- Track absolute deadline for each fiber
- Preempt lower-priority fibers on deadline pressure
- Admission control: reject if unschedulable
- Utilization bound checking (69.3% for RMS)
- Support for periodic and sporadic fibers

Integration with Phase 1

- **Fiber Scheduler**: Add deadline awareness to priorities
- **Graph Calls**: Use timers for timeout handling
- **Ring Buffers**: Tickless polling with adaptive backoff
- **Zero-Copy IPC**: Timeout support for blocking operations

Cross-Subsystem Integration

NUMA + Prefetch

- Prefetch from local NUMA node first
- Prefetch remote data only if necessary
- Use NUMA distance for prefetch priority
- Prefetch NUMA topology metadata

NUMA + Scheduler

- Schedule fibers on NUMA-local CPUs
- Migrate fibers to follow memory
- Balance load within NUMA nodes first
- Cross-NUMA migration as last resort

Prefetch + Scheduler

- Prefetch next fiber’s context before switch
- Prefetch timer wheel slots before expiration
- Prefetch deadline data for CRDS
- Adaptive prefetch based on scheduler state

All Three Together

- NUMA-aware timer allocation
- Prefetch timer wheel from local node
- Schedule timers on NUMA-local CPUs
- Tickless operation reduces NUMA traffic

Performance Monitoring

NUMA Metrics

- Local vs remote memory accesses
- Cross-NUMA traffic volume
- Migration frequency and cost
- NUMA node utilization balance

Prefetch Metrics

- TLB miss rate (target: <1%)
- Cache miss rate (L1/L2/L3)
- Prefetch accuracy (useful/total)
- Page fault rate reduction

Scheduler Metrics

- Timer IPI rate (target: <100/sec)
- Deadline miss rate (target: 0%)
- Context switch overhead
- Idle time in deep C-states

Scalability Analysis

Core Count Scaling

- Linear scaling to 128 cores
- Sub-linear scaling to 256 cores
- NUMA-aware design critical beyond 64 cores
- Per-CPU structures eliminate contention

Memory Scaling

- Support for 1TB+ memory
- Huge pages reduce page table overhead
- NUMA-aware allocation scales with nodes
- Attention-guided policy adapts to size

Timer Scaling

- O(1) timer operations regardless of count
- Hierarchical wheel handles millions of timers
- Per-CPU wheels eliminate global bottleneck
- Tickless operation scales to idle cores

Future Extensions (Phase 3-4)

Phase 3 Integration Points

- Hardware acceleration hooks
- DMA engine integration
- RDMA support for cross-NUMA
- GPU memory management

Phase 4 Integration Points

- Distributed NUMA across machines
- Remote memory access optimization
- Global scheduler coordination
- Cross-machine timer synchronization

References

1. Linux NUMA Implementation: `/sys/devices/system/node/`
2. Varghese & Lauck (1987): “Hashed and Hierarchical Timing Wheels”
3. Intel x86-64 Manual: PCID, INVPCID, Huge Pages
4. Liu & Layland (1973): “Rate Monotonic Scheduling”

5. Linux Kernel: `mm/huge_memory.c`, `kernel/time/timer.c`
6. Phase 1 Architecture: Lock-free rings, fiber scheduler, arena allocator

Conclusion

Phase 2's three-lever approach delivers comprehensive memory and scheduling optimizations that build naturally on Phase 1's foundations. By combining NUMA awareness, predictive prefetching, and tickless operation, we achieve significant performance gains while maintaining the simplicity and robustness required for production systems.

The architecture is designed for extensibility, with clear integration points for Phase 3-4 enhancements. All components are independently testable and can be enabled/disabled at runtime for gradual deployment.